

P3847R0: Lexical order for lambdas

Audience: EWG

S. Davis Herring <herring@lanl.gov>

Los Alamos National Laboratory

September 25, 2025

Problem

[`expr.prim.lambda.capture`]/10 specifies that members of a closure type are declared in an unspecified order, and /15 specifies that their initialization proceeds in that unspecified order. This wording predates *init-captures* entirely, and was perhaps principally intended to allow implementations not to track the order of (first) use of *implicit* captures. There is precedent in the out-of-order evaluation of *mem-initializers*, but (aside from those already being a known usability problem) the analogy fails because there is no canonical list of the members elsewhere whose order must be controlling.

This implementation freedom does not seem to provide useful benefits. The most direct possibility is minimizing padding, but the memory footprint of closures is rarely a concern. The other obvious possibility is general reordering optimizations; these are (still) permitted for function arguments, but the performance improvements are questionable.

However, it does mean that code like the following is unsafe:

```
void construction() {
    auto x = std::make_unique<int>();
    [value = *x,
     lifetime = std::move(x)] {};
}

struct last_call {
    std::function<void()> f;
    ~last_call() {f();}
};

void destruction() {
    auto x = std::make_unique<int>();
    const auto user = [&i = *x] {std::cout << i << '\n';};
    [lifetime = std::move(x),
     trigger = last_call{user}] {};
}
```

When value is initialized in construction, the move from x may have already occurred; when user is called in destruction, lifetime may have already been destroyed. The case shown here with *init-captures* is particularly surprising because they strongly resemble *init-declarators* (or *member-declarators* with default member initializers), which are ordered, but similar results can arise with non-reference *simple-captures* given copy constructors with side effects. Captures of either kind less resemble function parameters: both can be initialized in complicated manners from local variables, but parameters have a shorter lifetime and captured objects can have a longer one. It can easily happen that the precise amount of additional lifetime matters.

It is possible but unergonomic to force the ordering:

```
void construction() {
    auto x = std::make_unique<int>();
    [both = std::pair{*x, std::move(x)}] { // must be list-initialization
        auto &[value, _] = both; // or use both.first throughout
    };
}
```

Conveniently, implementations do not seem to make any use of this freedom: tests indicate that all major implementations define members for all non-reference explicit captures (*simple-captures* and *init-captures*) in order and then for all implicit captures in order of first capturing appearance. (MSVC erroneously considers the operand of `sizeof` to need a capture, but it still includes such captures in the same order.) Moreover, the Itanium ABI [intends](#) to specify this order for the layout of closure types.

Proposal

Specify, as a defect report, that all explicit captures are declared in the order in which they appear, as is existing practice. For simplicity, extend this to reference *simple-captures*, which can be observed only via implementation-defined reflection extensions ([`meta.reflection.member.queries`]/2). If EWG desires, either or both of the further existing practices of declaring all implicit captures later and in order of (capturing) appearance could be required, though the motivation to follow what is written in the source is weaker in that case. (The uses that cause implicit captures do in fact come after the explicit captures, but the *capture-default* of course comes first in the *capture-list*.)

It would then be safe to allow *init-captures* to refer to previous captures (in [`v, size0 = v.size()`], the captured `v` would have been initialized), but that is not proposed for obvious compatibility reasons.

Also repair the defect that [`expr.prim.lambda.capture`]/15 does not describe initialization for reference *init-captures* at all if, per /12, there are no members for them.

Wording

Relative to N5014.

[expr.prim.lambda.capture]

Change paragraph 10:

- [...]

For each entity captured by copy, an unnamed non-static data member is declared in the closure type. The declaration order of these members is consistent with the order of the explicit captures but otherwise unspecified. The type of such a data member is the referenced type if the entity is a reference to an object, an lvalue reference to the referenced function type if the entity is a reference to a function, or the type of the corresponding captured entity otherwise. A member of an anonymous union shall not be captured by copy.

Change paragraph 12:

- An entity is *captured by reference* if it is implicitly or explicitly captured but not captured by copy. It is unspecified whether additional unnamed non-static data members are declared in the closure type for entities captured by reference. If declared, such non-static data members shall be of literal type and be declared in an order consistent with the order of the explicit captures. [Example:

[...]

— *end example*] A bit-field or a member of an anonymous union shall not be captured by reference.

Change paragraph 15:

- When the *lambda-expression* is evaluated, the entities that are captured by copy are used to direct-initialize each corresponding non-static data member of the resulting closure object, and the non-static data members corresponding to the *init-captures* and the (reference) variables declared by *init-captures* without corresponding non-static data members are initialized as indicated by the corresponding *initializer* (which may be copy- or direct-initialization). (For array members, the array elements are direct-initialized in increasing subscript order.) These initializations are performed in an order consistent with the (partially unspecified) order in which the non-static data members are declared and with the order of explicit captures. [Note: This ensures that the destructions will occur in the reverse order of the constructions. — *end note*]